

Density-Based Traffic Signal Control

Installation Guide — Hardware and Software

Deployment manual

2026-06-15

Contents

1	Overview	1
2	Bill of Materials	1
3	Part A — Workstation (Software)	2
3.1	Install Ubuntu 22.04	2
3.2	Install system dependencies	2
3.3	Install uv and clone the repository	2
3.4	Create the virtual environment and install requirements	2
3.5	Confirm the model weights are present	3
4	Part B — Camera Network	3
4.1	Wire the cameras	3
4.2	Give the workstation a static IP on the camera subnet	3
4.3	Make the static IP persistent	3
5	Part C — 2.4 GHz Hotspot for the ESP32 Lights	4
5.1	Firmware-expected credentials	4
5.2	Force the hotspot to 2.4 GHz (Android)	4
5.3	Connect the workstation to the same hotspot	4
6	Part D — ESP32 Traffic Lights (Hardware + Firmware)	4
6.1	Wiring	4
6.2	Install the Arduino IDE and ESP32 support	5
6.3	Find each board's MAC address	5
6.4	Flash the two sketches	5
6.5	How the lights coordinate	5
7	Part E — Running the System	5
7.1	Smoke-test the light link (no cameras needed)	5
7.2	Launch the application	6
7.3	Load a configuration	6
7.4	Tuning detection and emergency behaviour	6

1 Overview

The system has three cooperating parts:

1. **Cameras** — four IP cameras stream video over RTSP on a wired network.
2. **Workstation** — a Linux/GPU machine runs the PySide6 application, detects vehicles with YOLO11, decides the phase for the two lights (vertical and horizontal lanes) and publishes commands over MQTT.
3. **Traffic lights** — two ESP32 boards, one per lane, subscribe to their MQTT topic, drive the red/yellow/green outputs, and hand the green phase to each other over ESP-NOW.

Data flow:

Cameras → Workstation (YOLO + logic) → MQTT broker (broker.hivemq.com) → ESP32 lights ↔ ESP-NOW

The MQTT broker is the public `broker.hivemq.com` on port 1883, so both the workstation and the ESP32 boards need Internet access. The cameras live on a separate, isolated wired network.

2 Bill of Materials

Item	Qty	Notes
GPU workstation	1	Ubuntu 22.04, NVIDIA GPU (RTX 2060 or better)
IP cameras (RTSP)	4	Hikvision-compatible, PoE
PoE switch	1	To power and network the cameras
USB-to-Ethernet adapter	1	Dedicated NIC for the camera network
Ethernet cables	5	Cameras + uplink
ESP32 dev boards	2	One per lane (vertical, horizontal)
Traffic-light LEDs/relays	2 sets	Red, yellow, green per lane
Resistors / relay board	—	For LEDs or to switch mains lamps
2.4 GHz Wi-Fi hotspot	1	Phone hotspot is fine (see §5)
Jumper wires, breadboard / PCB	—	For ESP32 wiring

3 Part A — Workstation (Software)

3.1 Install Ubuntu 22.04

Install Ubuntu 22.04 LTS on the GPU machine. During installation, tick “*Install third-party software for graphics and Wi-Fi hardware...*” so the NVIDIA driver is set up automatically. After first boot, confirm the GPU is visible:

```
nvidia-smi
```

You should see your GPU and driver version. If the command is missing, install the recommended driver and reboot:

```
sudo ubuntu-drivers autoinstall
sudo reboot
```

3.2 Install system dependencies

The capture pipeline uses GStreamer through PyGObject, and the test scripts use the Mosquitto client tools. Install them system-wide:

```
sudo apt update
sudo apt install -y git curl \
  python3-gi python3-gi-cairo gir1.2-gstreamer-1.0 gir1.2-gst-plugins-base-1.0 \
  gstreamer1.0-plugins-good gstreamer1.0-plugins-bad gstreamer1.0-plugins-ugly \
  gstreamer1.0-libav gstreamer1.0-tools \
  mosquitto-clients
```

The application imports the system GStreamer bindings from `/usr/lib/python3/dist-packages`, so `python3-gi` and the GStreamer plugins above must be present even though the rest of the app runs in a virtual environment.

3.3 Install uv and clone the repository

```
curl -LsSf https://astral.sh/uv/install.sh | sh
# restart the shell or: source $HOME/.local/bin/env

cd ~
mkdir -p project && cd project
git clone https://github.com/lukasdante/density-traffic-control.git
cd density-traffic-control
```

If `git` prompts for a username/email, set the global config it asks for, then re-run the `git clone` line.

3.4 Create the virtual environment and install requirements

```
uv venv env
source env/bin/activate      # prompt now shows (env)
uv pip install -r requirements.txt
```

This pulls in PyTorch (CUDA 12 build), Ultralytics, OpenCV, PySide6 and the other dependencies. Verify CUDA is available to PyTorch:

```
python -c "import torch; print('CUDA:', torch.cuda.is_available())"
```

It should print `CUDA: True`. If it prints `False`, re-check the NVIDIA driver (`nvidia-smi`).

3.5 Confirm the model weights are present

The detector loads `model/runs/yolo11-vehicles2/weights/best.pt`. Confirm it exists:

```
ls model/runs/yolo11-vehicles2/weights/
```

If the weights are not in the clone, copy them into that path before running.

4 Part B — Camera Network

4.1 Wire the cameras

Connect the four cameras and the workstation's USB-to-Ethernet adapter to the PoE switch. The cameras use the fixed addresses below.

Camera	IP address	Lane (direction)
Camera A	192.168.1.64	vertical
Camera B	192.168.1.65	vertical
Camera C	192.168.1.66	horizontal
Camera D	192.168.1.67	horizontal

Camera credentials (RTSP):

- Username: `admin`
- Password: `ub-traffic-light`
- RTSP port: 554 (HTTP 80, HTTPS 443)

4.2 Give the workstation a static IP on the camera subnet

Find the name of the Ethernet adapter:

```
ip link show
```

Look for the wired device, typically named like `enx00e04c36131a`. Assign a *temporary* static IP to test connectivity (replace `<name>`):

```
sudo ip addr add 192.168.1.200/24 dev <name>
ping 192.168.1.64 # repeat for .65 .66 .67
```

4.3 Make the static IP persistent

The `ip addr add` above is lost on reboot. Make it permanent with NetworkManager (replace `enx00e04c36131a` with your adapter name):

```
sudo nmcli connection delete enx00e04c36131a
sudo nmcli connection add type ethernet con-name "CameraNet" \
    ifname enx00e04c36131a ipv4.method manual ipv4.addresses 192.168.1.200/24
sudo nmcli connection up "CameraNet"
ip addr show enx00e04c36131a
```

Verify you can still ping all four cameras after this step.

5 Part C — 2.4 GHz Hotspot for the ESP32 Lights

The ESP32 only supports 2.4 GHz Wi-Fi. The boards (and the broker they talk to) need Internet, so the simplest setup is a phone hotspot forced to the 2.4 GHz band, configured with the credentials the firmware expects.

5.1 Firmware-expected credentials

The shipped firmware connects to:

- SSID: `ub-hs-2.4G`
- Password: `ub-hs-0211`

Either configure your hotspot to use exactly these values, *or* change `WIFI_SSID` / `WIFI_PASS` in both ESP sketches to match your hotspot (see Part D).

5.2 Force the hotspot to 2.4 GHz (Android)

1. Settings → *Network & Internet* → *Hotspot & tethering* → *Wi-Fi hotspot*.
2. Set *Hotspot name* to `ub-hs-2.4G` and *Password* to `ub-hs-0211`.
3. Set *AP Band* to *2.4 GHz* (do not use “Auto” or 5 GHz).
4. Enable the hotspot.

On iPhone, enable *Personal Hotspot* and turn on *Maximize Compatibility* to force 2.4 GHz; note that iOS does not let you set an arbitrary SSID, so you will need to edit the firmware credentials to match the iPhone’s hotspot name and password.

5.3 Connect the workstation to the same hotspot

The workstation publishes to `broker.hivemq.com` over the Internet, so connect its Wi-Fi to the same hotspot (the wired **CameraNet** interface stays dedicated to the cameras). The machine therefore has two networks at once: wired for cameras, Wi-Fi for Internet/MQTT.

6 Part D — ESP32 Traffic Lights (Hardware + Firmware)

6.1 Wiring

Each ESP32 drives one light. The outputs are **active-LOW** (a pin driven LOW turns its lamp *on*):

Signal	ESP32 GPIO
Red	27
Yellow	26
Green	25

For LED indicators, wire each GPIO through a current-limiting resistor (e.g. 220–330 Ω) to the LED, LED cathode to GND. To switch real traffic lamps, drive an opto-isolated relay board from these pins and observe the active-LOW polarity (most relay boards are also active-LOW, which matches). Power the ESP32 from a stable 5 V USB supply.

6.2 Install the Arduino IDE and ESP32 support

1. Install the Arduino IDE (2.x).
2. *File* → *Preferences* → *Additional Boards Manager URLs*, add:
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
3. *Tools* → *Board* → *Boards Manager*, install *esp32 by Espressif Systems*.
4. *Tools* → *Manage Libraries*, install *PubSubClient* by Nick O’Leary. (WiFi and `esp_now` ship with the ESP32 core.)

6.3 Find each board's MAC address

The two boards reference each other by MAC, so record them first. Open either sketch, select *Tools* → *Board* → *ESP32 Dev Module*, pick the serial *Port*, upload, then open *Serial Monitor* at 115200 baud and reset the board. It prints its MAC, e.g. [Wi-Fi] Connected, IP: ...| MAC: 88:57:21:AD:7C:34.

6.4 Flash the two sketches

There is one sketch per lane. Each must (a) subscribe to its own MQTT topic and (b) point its ESP-NOW `peerAddress` at the *other* board's MAC.

- **Vertical light** — `esp/bidirectional_mqtt_vertical/bidirectional_mqtt_vertical.ino`
Topic: `ub-traffic-light/signals/vertical`; `peerAddress` = MAC of the *horizontal* board.
- **Horizontal light** — `esp/bidirectional_mqtt_horizontal/bidirectional_mqtt_horizontal.ino`
Topic: `ub-traffic-light/signals/horizontal`; `peerAddress` = MAC of the *vertical* board.

In each sketch confirm/adjust the configuration block at the top:

```
const char* WIFI_SSID = "ub-hs-2.4G";           // match your hotspot
const char* WIFI_PASS = "ub-hs-0211";          // match your hotspot
const char* MQTT_SERVER = "broker.hivemq.com";  // already set
const int   MQTT_PORT  = 1883;
const char* MQTT_TOPIC = "ub-traffic-light/signals/vertical"; // or /horizontal
uint8_t peerAddress[] = { /* MAC of the OTHER ESP32 */ };
```

Upload each sketch to its board. The `bidirectional_mqtt` sketch (no suffix) is a reference MQTT→ESP-NOW forwarder and is *not* needed for the two-light deployment.

6.5 How the lights coordinate

The workstation only ever publishes "red" to the lane that should lose priority. That board turns yellow for 3s, then tells its peer to go "green" over ESP-NOW, then turns itself red. Each board also enforces a hardware safety cap (`MAX_GREEN_SECONDS` = 90) so no lamp can stay green indefinitely if a command is missed.

7 Part E — Running the System

7.1 Smoke-test the light link (no cameras needed)

With both ESP32 boards powered and on the hotspot, publish a command from the workstation and watch the lamps and Serial Monitor:

```
# from the repo root, with the venv active
./vertical_test.sh      # publishes "red" to the vertical topic
./horizontal_test.sh    # publishes "red" to the horizontal topic

# or directly:
python gui/publisher.py vertical red
mosquitto_pub -h broker.hivemq.com -p 1883 \
  -t "ub-traffic-light/signals/vertical" -m "red"
```

You should see the addressed lane go yellow then red, and the other lane go green.

7.2 Launch the application

```
cd ~/project/density-traffic-control
source env/bin/activate
python gui/main.py
```

7.3 Load a configuration

Ready-made configs live in `configs/`:

- `all_cameras.json` — the four live RTSP cameras.
- `proxy_cameras.json` / `focus_first_camera.json` — local video files for testing without hardware (`use_videos: true`).

Load the appropriate config from the GUI, then start inference. The grid shows each camera, detected vehicles, and (when enabled in the OSD settings) the current signal phase per lane.

7.4 Tuning detection and emergency behaviour

Key knobs in the config `experimental_settings` block:

- `confidence` (default 0.30) — detection threshold for vehicles.
- `tile_w` / `tile_h` — per-camera inference resolution; lower values trade accuracy for speed on weaker GPUs.
- `emergency_confidence` (default 0.55) — minimum confidence for an ambulance/firetruck to count toward an emergency.
- `emergency_frames` (default 5) — consecutive cycles an emergency must persist before that lane is forced green.

To watch emergency detections, enable the `ambulance` and `firetruck` bounding boxes in the display settings.

8 Troubleshooting

- `torch.cuda.is_available()` is **False** — NVIDIA driver not loaded; run `nvidia-smi`, reinstall the driver, reboot.
- No module named `gi` — install `python3-gi` and the GStreamer packages (§A.2); they must be present system-wide.
- **Cameras unreachable** — confirm the CameraNet static IP with `ip addr show` and `ping 192.168.1.64`; check the PoE switch.
- **ESP32 won't connect to Wi-Fi** — the hotspot must be 2.4 GHz and the SSID/password must match the firmware exactly; watch the Serial Monitor at 115200.
- **Lights don't switch** — verify both boards are online (Serial Monitor shows *Subscribed*), that each `peerAddress` is the *other* board's MAC, and that a manual `mosquitto_pub` toggles them.
- **Lamp polarity inverted** — the outputs are active-LOW; swap LED wiring or relay logic accordingly.